



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

CSA4712 – DEEP LEARNING

CO1 – ASSESSMENT 1

MATHEMATICAL FOUNDATIONS OF CROP-DISEASE IMAGE CLASSIFICATION

Student Name	B Akshaya
Registration Number	192424336
Submission Date	10.10.2026
Scenario	AgriVision crop-disease screening

Assignment overview

This assignment explains the mathematical and machine-learning choices behind a crop-disease screening system. Each task is presented as connected technical discussion with a worked example or data table, a code excerpt and a notebook-style plot panel. The code excerpts are templates to adapt and execute in Google Colab; sample charts are illustrative unless explicitly replaced by results from the executed notebook. This distinction keeps the report reproducible and avoids presenting unmeasured values as experimental evidence.

TASK 1 — Problem structure and data representation

AgriVision is a supervised multi-class image-classification system. A photograph is the input and a disease category is the output; therefore, class definitions, image quality and class balance constrain the solution. Overall accuracy is insufficient when rare diseases matter, so macro-F1, per-class recall and a confusion matrix should accompany it. A grayscale 64×64 image is a matrix with 4,096 scalar pixels; flattening creates a 4,096-element vector but removes explicit spatial neighbourhoods. A batch of 32 images has shape (32, 64, 64, 1), while flattened features have shape (32, 4096). Labels can be integer IDs or one-hot vectors. Resizing reduces cost but may erase small lesions; grayscale removes colour cues; normalization to [0,1] improves scale; and splitting by plant/source helps prevent duplicate leakage. L^1 measures total absolute magnitude, L^2 measures Euclidean magnitude, and L^∞ captures the largest single deviation. The representation should preserve disease information while meeting memory and latency limits.

Sample tensor shapes

Method and analysis. For example, converting uint8 pixels to floating point and dividing by 255 maps pixel intensities into [0,1]. This gives the model a predictable scale. If images are collected from the same plant or video, grouping them before splitting is important because almost identical frames in training and testing can exaggerate performance.

Interpretation. A useful report records original image size, resized image size, channel count, number of classes and examples per class. The class-distribution plot helps identify imbalance before model training. If a rare disease has very few images, macro-averaged metrics and class-specific recall should receive particular attention.

Object	Shape	Meaning
Image	64×64	4,096 pixel values
Batch	$32 \times 64 \times 64 \times 1$	32 grayscale images
Flattened	$32 \times 4,096$	One vector per image

```
images = images.astype('float32') / 255.0
X = images.reshape(len(images), -1)
print('Batch shape:', images.shape)
print('Feature shape:', X.shape)
```



TASK 2 — Feature-space analysis using linear algebra

Pixel features are correlated because leaf images share shape, lighting, background and lesion structure. After centering the training matrix X , PCA uses $C = X^T X / (n - 1)$; eigenvectors identify principal directions and eigenvalues quantify variance in those directions. For the same covariance convention, $\sigma_i^2 = \lambda_i^2 / (n - 1)$, where λ_i are singular values. Explained variance ratio is $\lambda_i^2 / \sum \lambda_i^2$. A steep spectrum indicates redundancy; a flat spectrum means compression can discard more information. For k components, reconstruction is $X_{\text{recon}} = XW_k W_k^T$ and error generally falls as k rises, while storage and compute increase. Select k using a reconstruction curve and validation macro-F1 rather than a fixed variance threshold alone. Fit PCA only on training data. A hand example $C = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ has eigenvalues 3 and 1, so the first component explains $3/4 = 75\%$ of variance. Full covariance decomposition is expensive for 4,096 dimensions; randomized or incremental SVD may be preferable.

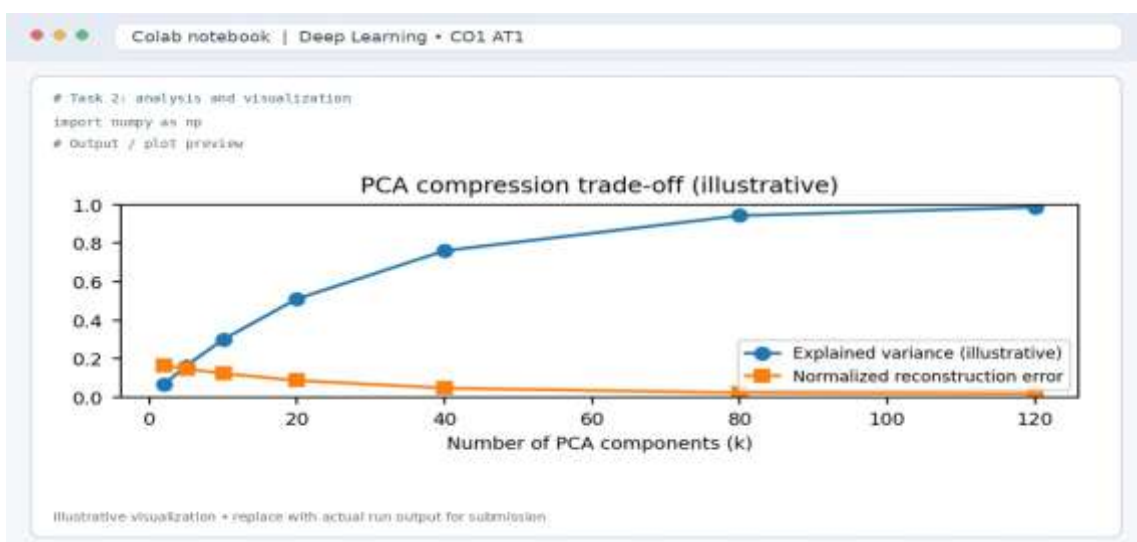
Small worked example

Method and analysis. PCA is a linear projection: the first component maximizes projected variance, and each later component maximizes remaining variance subject to orthogonality. The eigenvectors form the projection matrix. Centering is essential because PCA describes variation around the mean rather than absolute pixel intensity.

Interpretation. The component count should be chosen on a validation set. Explained variance gives a compression summary, while classification metrics test whether class-relevant information survived. PCA may preserve lighting or background variation because these can dominate total variance; a high explained-variance percentage alone does not prove that disease recognition is good.

Eigenvalue	Explained variance	Interpretation
3	75%	First principal direction
1	25%	Second principal direction
Total = 4	100%	Total variance

```
from sklearn.decomposition import PCA
pca = PCA(n_components=0.95,
random_state=42) X_train_pca =
pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
print('Retained variance:', pca.explained_variance_ratio_.sum())
```



TASK 3 — Probability and information-theoretic uncertainty

Class entropy $H(p) = -\sum p(c) \log p(c)$ measures uncertainty in the label distribution. It is low when one class dominates and reaches $\log(C)$ bits for C equally likely classes; for three balanced classes the maximum is 1.585 bits. Cross-entropy $H(p, q) = -\sum p(c) \log q(c)$ penalizes predictions that assign low probability to the true class. KL divergence $DKL(p \parallel q) = \sum p(c) \log[p(c)/q(c)]$ measures the directed mismatch between distributions and is non-negative. Because $H(p, q) = H(p) + DKL(p \parallel q)$, minimizing cross-entropy also minimizes KL when the true distribution is fixed. KL is asymmetric: for $p = [0.9, 0.1]$ and $q = [0.5, 0.5]$, $DKL(p \parallel q) \approx 0.368$ nats while $DKL(q \parallel p) \approx 0.511$ nats. Imbalanced labels require macro-F1 and per-class recall; calibration and Brier score help evaluate confidence. Label noise can increase apparent uncertainty and should be considered when interpreting loss.

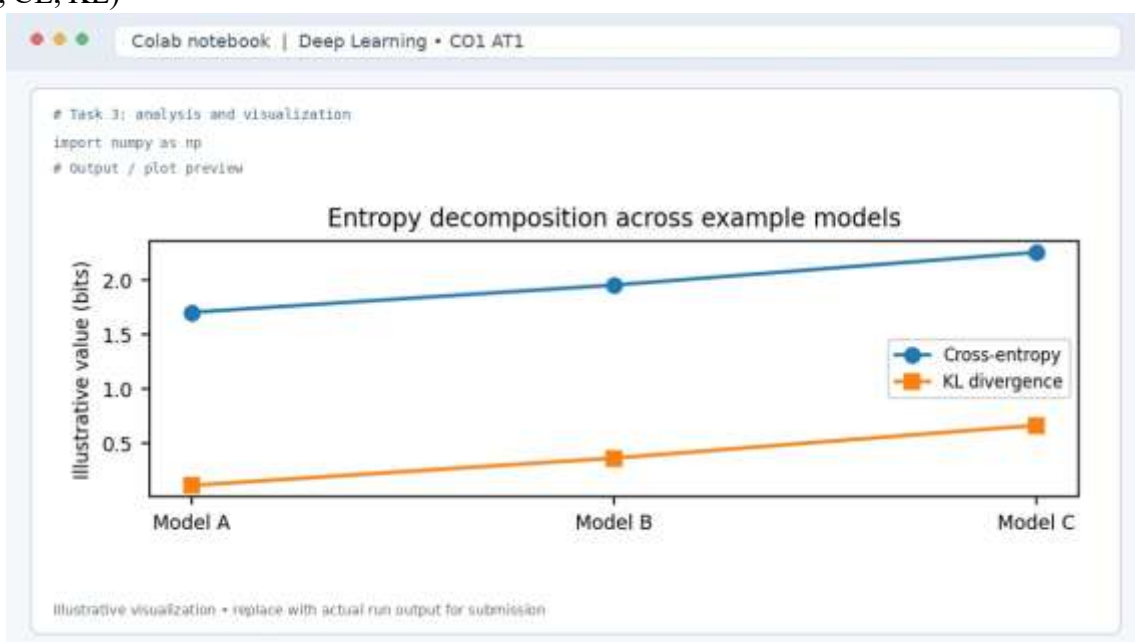
Illustrative class distribution

Method and analysis. Probability outputs are not merely class labels: they describe the model's confidence across possible classes. A prediction such as $[0.70, 0.20, 0.10]$ is more informative than the class ID alone, but its confidence is useful only if it is reasonably calibrated.

Interpretation. Use the same logarithm base when comparing entropy, cross-entropy and KL divergence. If a predicted probability is zero for a class with positive true probability, cross-entropy can become infinite; in code, stable library functions or a small numerical safeguard may be needed. The safeguard should be documented.

Metric	Definition	Use
Entropy	$-\sum p \log p$	Label uncertainty
Cross-entropy	$-\sum p \log q$	Prediction loss
KL divergence	$\sum p \log(p/q)$	Distribution mismatch

```
from scipy.stats import
entropy H = entropy(p,
base=2)
CE = -np.sum(p * np.log2(q +
1e-12)) KL = entropy(p, q,
base=2)
print(H, CE, KL)
```



TASK 4 — Numerical computation and stability

Softmax is $\exp(z_i) / \sum \exp(z_i)$. For logits $[1000, 999, 995]$, direct exponentiation can overflow to infinity and yield an undefined infinity/infinity ratio. Very negative logits can underflow to zero and trigger $\log(0)$. Since softmax is invariant to a shared shift, subtract $m = \max(z)$ before exponentiation. The shifted logits $[0, -1, -5]$ have finite exponentials and yield probabilities approximately $[0.7275, 0.2676, 0.0049]$. The

log-sum-exp identity $\log \sum \exp(z_i) = m + \log \sum \exp(z_i - m)$ gives stable log probabilities without changing the mathematics. A covariance condition number $\kappa = \sigma_{\max} / \sigma_{\min}$ describes scale disparity; a large value can make gradient descent zigzag or converge slowly. Scaling, justified PCA and regularization can help conditioning, although compression can remove signal. Stable library loss functions and monitoring for NaN, infinity and exploding gradients are recommended.

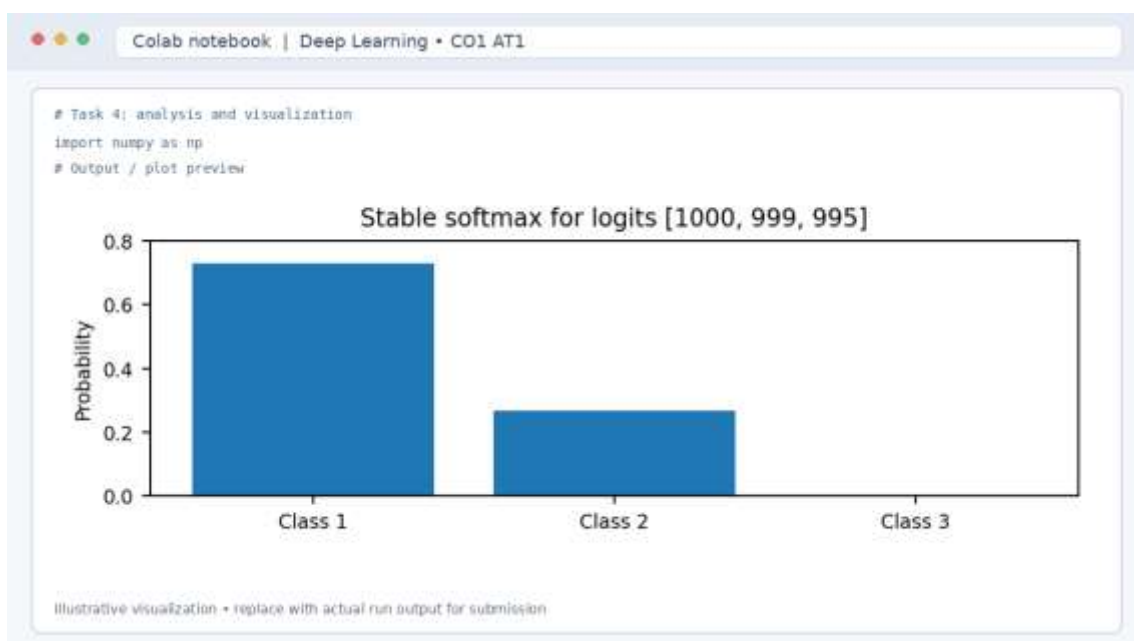
Softmax numerical example

Method and analysis. Floating-point numbers have a finite representable range. Direct exponentiation of very large logits may overflow, while exponentiation of very negative values may underflow to zero. These are numerical representation problems rather than flaws in the softmax definition.

Interpretation. Subtracting the maximum logit changes every logit by the same constant and therefore leaves softmax probabilities unchanged. This simple transformation keeps exponentials bounded by one. The stable implementation should be used in training and inference, and the same principle underlies stable log-softmax and cross-entropy functions.

Issue	Cause	Stable solution
Overflow	Very large logits	Subtract max logit
Underflow	Very negative logits	Use stable log-softmax
Ill-conditioning	Unequal scales	Scale / regularize

```
z = np.array([1000., 999., 995.])
z = z - np.max(z)
p = np.exp(z) / np.exp(z).sum()
print(np.round(p, 4))
```



TASK 5 — Comparison of baseline learning strategies

Logistic regression is a parametric linear classifier with efficient training and a relatively strong simplicity bias. k-nearest neighbours is non-parametric and builds locally flexible boundaries, but distance becomes less informative in 4,096 dimensions and prediction can be expensive. Random Forest models nonlinear interactions and is less sensitive to feature scaling, but may need many splits to capture fine image textures. PCA can reduce noise and speed up k-NN, yet PCA maximizes variance rather than class separation; low-variance disease cues may disappear. Logistic regression may benefit from compression and regularization, while Random Forest can lose details if too few components remain. Use the same stratified split and tune on validation data only. Compare macro-F1, per-class recall, confusion matrices, training time, inference latency and model size. The table and chart are synthetic examples for illustrating reporting format, not measured results from PlantVillage.

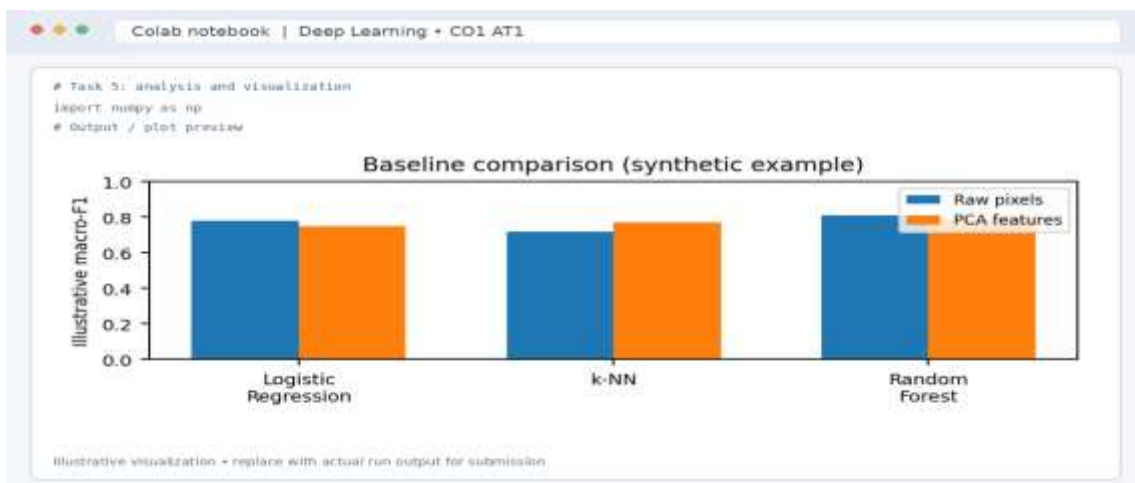
Example reporting table — replace with measured values

Method and analysis. A fair baseline study uses the same train, validation and test partitions for each model. Any scaler or dimensionality reduction step must be fitted only on the training partition. Hyperparameters such as k for k-NN, regularization strength for logistic regression and tree count or depth for Random Forest should be selected using validation data.

Interpretation. The table should include the feature representation because raw-pixel and PCA-based results answer different questions. Report macro-F1 and class recall with accuracy, and record runtime and model size. Any example scores in a demonstration table must be replaced with measured values before drawing a conclusion about the best model.

Model	Strength	Limitation
Logistic regression	Fast, simple	Linear boundary
k-NN	Flexible local boundary	High-dimensional distance
Random Forest	Nonlinear interactions	Can be large

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import
StandardScaler from sklearn.linear_model
import LogisticRegression
model = make_pipeline(StandardScaler(),
LogisticRegression(max_iter=1000)) model.fit(X_train, y_train)
print('Macro-F1:', f1_score(y_test, model.predict(X_test), average='macro'))
```



TASK 6 — Capacity, overfitting and underfitting

Capacity is the complexity of patterns a model can represent. A degree-one model may underfit nonlinear structure, while a deep tree or highly flexible polynomial may memorize training examples. In k-NN, very large k smooths the boundary and can underfit; k=1 is sensitive to noise and can overfit. Underfitting typically produces high training and validation error. Overfitting produces low training error but a larger validation error. A useful model is near the minimum validation error with a manageable generalization gap. Plot errors across polynomial degree, tree depth, k or regularization strength and explain why the curve changes. Hyperparameters must be selected using a validation set or cross-validation. Repeatedly selecting settings from test results leaks information and biases the final test score optimistically. The shown capacity curve is a synthetic illustration of the expected shape, not a result from the crop dataset.

Capacity/error illustration

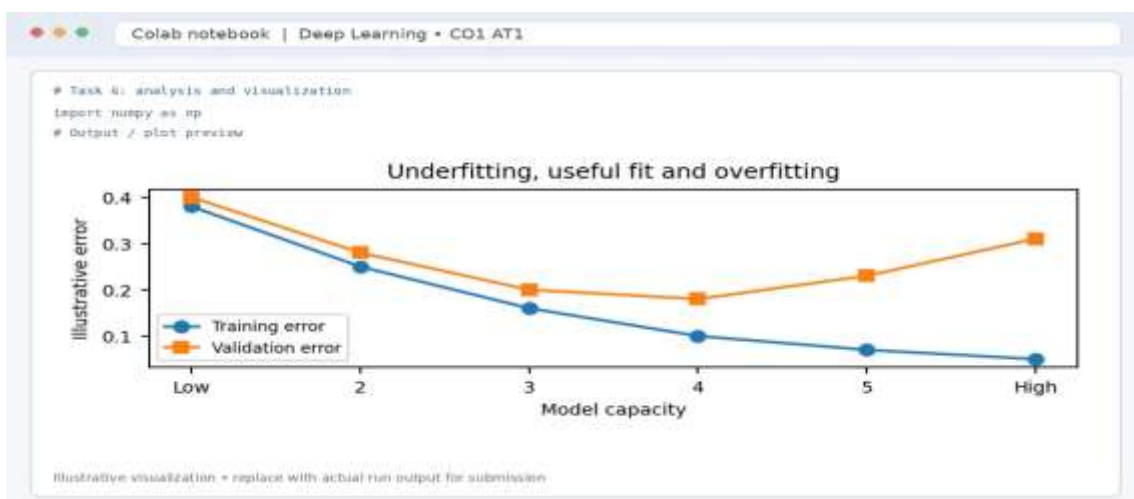
Method and analysis. Capacity can be varied systematically using polynomial degree, tree depth, number of neighbours or regularization strength. The model is fitted repeatedly under each setting and training and validation errors are stored. Plotting both curves makes the generalization gap visible and helps distinguish insufficient flexibility from excessive flexibility.

Interpretation. The best setting is usually selected near the minimum validation error, not the minimum training error. If validation error rises while training error continues to fall, the model is fitting sample-specific patterns. More data, regularization, simpler architecture or better data quality may reduce this behaviour.

Capacity	Training error	Validation error
Low	High	High: underfitting
Moderate	Lower	Lowest: useful fit
High	Very low	Rises: overfitting

for depth in [2, 4, 8, 16]:

```
model = DecisionTreeClassifier(max_depth=depth,  
random_state=42) model.fit(X_train, y_train)  
train_err = 1 - model.score(X_train,  
y_train) val_err = 1 -  
model.score(X_val, y_val)  
print(depth, train_err, val_err)
```



TASK 7 — Bias–variance decomposition and estimation

Expected squared prediction error can be decomposed into bias², variance and irreducible noise under standard assumptions. Bias is the difference between average prediction and the true function; variance measures sensitivity to the training sample; noise remains even with an ideal estimator. A bootstrap experiment resamples the training set with replacement at least ten times, fits low- and high-capacity models, and stores predictions at fixed evaluation points. Estimate bias² from the squared difference between mean prediction and known target, and variance from prediction spread. In real crop data, the true target function and noise level are not known exactly, so the decomposition is approximate and its residual should be discussed. For class proportions, MLE is s/n . With a $\text{Beta}(\alpha, \beta)$ prior, posterior is $\text{Beta}(\alpha + s, \beta + n - s)$.

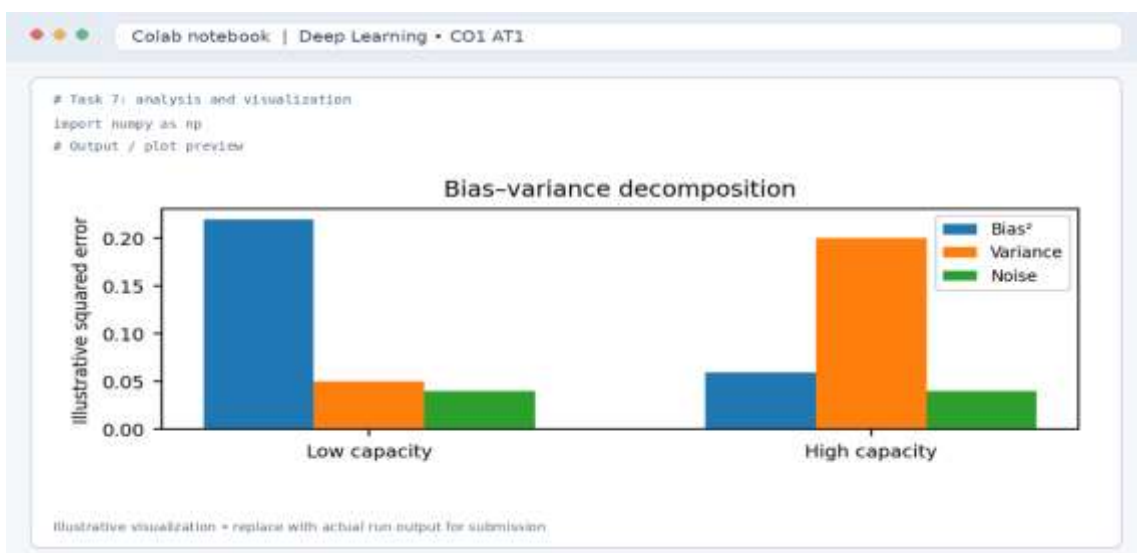
For $s=6, n=10$ and $\text{Beta}(2, 2)$, $\text{MLE}=0.60$ and posterior mean $= 8/14 \approx 0.571$. The prior influence declines as sample size increases. Illustrative decomposition

Method and analysis. Bootstrap replicas are useful because they approximate how the fitted model would vary under alternative samples from the same population. Predictions should be recorded on a fixed evaluation set so the variance estimate compares the same inputs across replicas.

Interpretation. The exact bias–variance decomposition relies on assumptions about squared-error prediction and a defined target function. For real image classification, it is usually an explanatory framework rather than a directly observable exact decomposition. The report should distinguish estimated quantities from known quantities and describe the bootstrap procedure.

Term	Meaning	How estimated
Bias ²	Systematic error	Mean prediction vs target
Variance	Sample sensitivity	Spread across replicas
Noise	Unavoidable uncertainty	Requires assumptions

```
predictions =  
[]  
for b in  
range(30):  
    idx = rng.choice(len(X_train), len(X_train),  
replace=True) model.fit(X_train[idx],  
y_train[idx])  
    predictions.append(model.predict(X_eval))  
# Compare prediction spread across bootstrap replicas
```



TASK 8 — Gradient-based optimization

For softmax regression, $z = Wx + b$ and $p = \text{softmax}(z)$. With one-hot target y , cross-entropy is $L = -\sum y_i \log p_i$. The logit gradient is

$\frac{\partial L}{\partial z} = p - y$, giving $\frac{\partial L}{\partial W} = (p - y)x^T$ and $\frac{\partial L}{\partial b} = p - y$ for one example. Batch gradient descent updates from all training examples and tends to have a smoother path but greater cost per update. SGD updates per example and is noisy; mini-batch SGD balances noise and throughput. A low learning rate is stable but slow, a moderate rate may converge efficiently, and an excessive rate can oscillate or diverge. Compare at least three learning rates using the same split and initialization; plot loss against epochs and record elapsed time. Feature covariance conditioning can influence convergence because gradients may vary strongly by direction. The plotted paths are synthetic illustrations, so actual experiment curves should replace them when code is run.

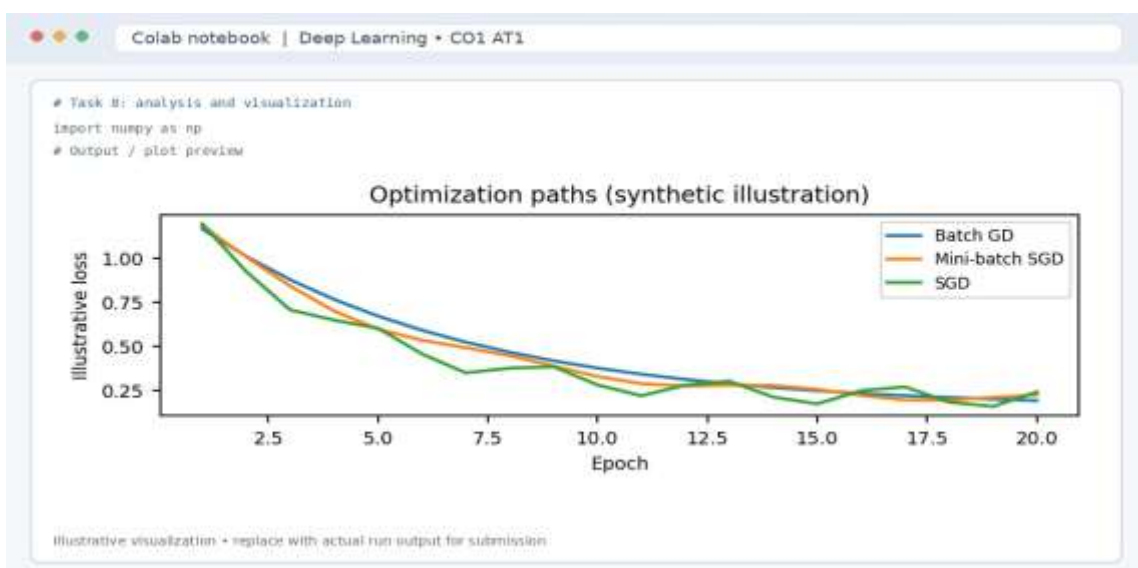
Optimization comparison

Method and analysis. A training curve should be recorded at consistent intervals and use the same dataset split for each optimizer. Learning-rate experiments should begin from comparable initial weights; otherwise, differences may come from initialization rather than the learning rate.

Interpretation. Batch gradient descent has a low-noise gradient but expensive updates. SGD gives frequent, noisy updates, while mini-batches provide a practical compromise on modern hardware. Convergence should be judged by loss, validation behaviour and time—not only by the number of epochs.

Optimizer	Update data	Behaviour
Batch GD	All samples	Smooth, expensive
SGD	One sample	Noisy, frequent
Mini-batch	Small batch	Practical compromise

```
for lr in [0.001, 0.01, 0.1]:  
    # Train with identical split and initial  
    weights history =  
    train_model(learning_rate=lr)  
    plt.plot(history['loss'], label=f'lr={lr}')  
plt.legend(); plt.xlabel('Epoch'); plt.ylabel('Loss')
```



TASK 9 — Experimental performance across dataset scales

Evaluate the selected baselines at 1,000, 5,000, 10,000 and full training-image scales. Keep a common held-out test set and preserve class proportions in each subset. Record accuracy, macro precision, macro recall, macro-F1, confusion matrix, training time, per-image latency, parameter count, file size and peak memory. Accuracy measures overall correctness but may hide minority-class failures; precision measures the reliability of predicted classes, recall measures detection of true examples, and F1 balances the two. Plot macro-F1 against training size and latency against PCA dimension. Improvement may plateau if labels are noisy or features are weak. Rare-class recall may improve more slowly than majority-class accuracy. Repeat timing runs and document hardware. The included learning curve uses illustrative values only; the results table is a template and should be filled with actual model evaluations before submission.

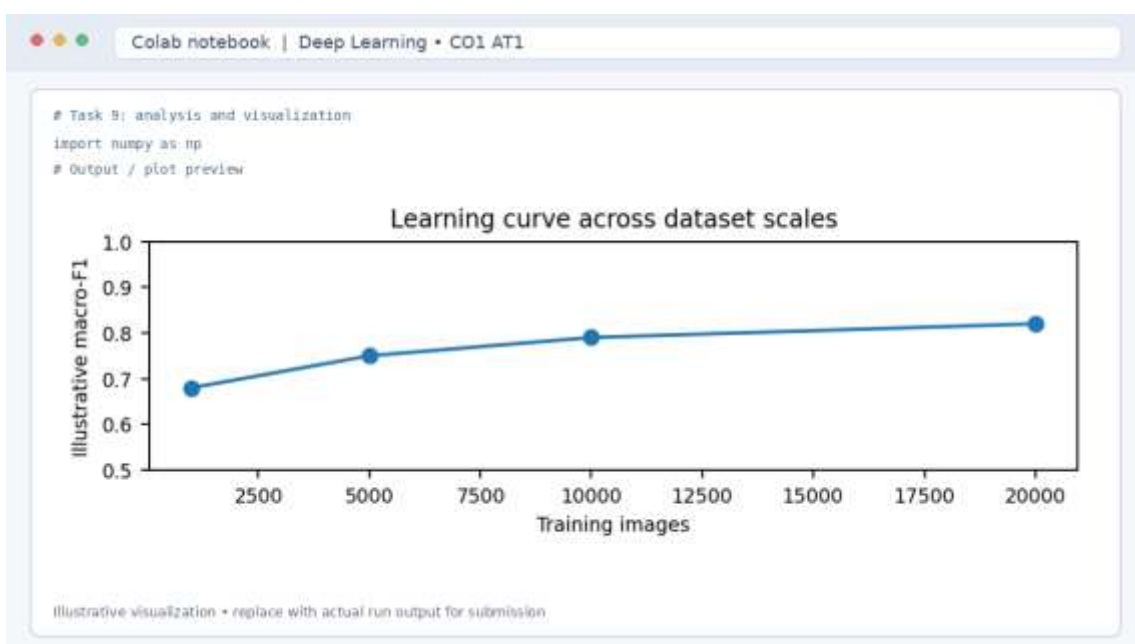
Results table template

Method and analysis. To study data scale, construct stratified subsets from the training partition while preserving the same test set. Use a fixed random seed or repeated seeds and document whether subsets are nested. This makes the effect of additional data easier to interpret.

Interpretation. The learning curve can reveal diminishing returns: early increases in data may improve generalization substantially, while later increases produce smaller gains. Timing and memory measurements should be taken under the same hardware and software conditions. If the full dataset is unavailable, that limitation must be stated.

Scale	Record	Purpose
1,000 images	Macro-F1 / time	Small-data baseline
5,000 images	Macro-F1 / time	Growth trend
Full dataset	Metrics / memory	Final comparison

```
for n in [1000, 5000, 10000, len(X_train)]:  
    subset = stratified_subset(X_train, y_train, n, random_state=42)  
    model.fit(subset.X, subset.y)  
    pred = model.predict(X_test)  
    print(n, f1_score(y_test, pred, average='macro'))
```



TASK 10 — Theory versus experimental behaviour

PCA preserves directions with high variance, but variance is not identical to class-discriminative information. Consequently, explained variance and classification performance need not fall at the same rate. If accuracy collapses at small k , discarded low-variance components may encode subtle lesions; stable accuracy suggests redundancy was removed. Conditioning theory predicts slower optimization for a covariance matrix with a large condition number. Compare scaled and unscaled data using identical splits, initialization and learning-rate schedules. Bias–variance theory predicts high error at low capacity and a growing generalization gap at excessive capacity. Compare these expectations with capacity curves and bootstrap estimates. Disagreements can result from finite samples, noisy labels, data leakage, duplicate images, optimizer settings or hardware variation. A strong report states the theoretical expectation, observed measurement, agreement and likely cause. The figure gives an illustrative comparison layout, not empirical findings.

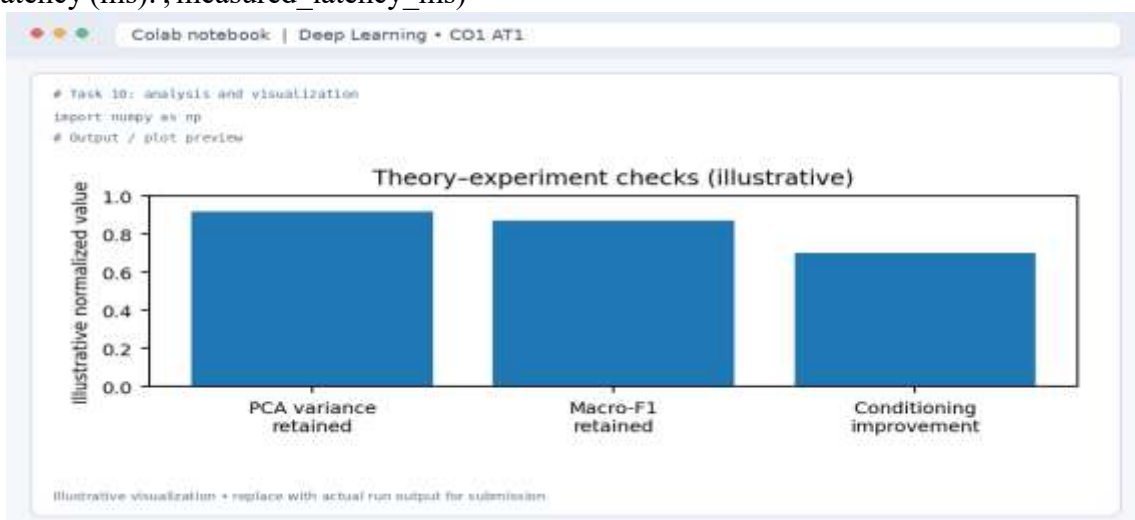
Interpretation table

Method and analysis. A theory-to-experiment comparison is strongest when it states a prediction before looking at the result. For example, PCA is expected to reduce dimension and memory, but its effect on macro-F1 is empirical. A clear comparison lists the predicted trend, observed value and a reason for agreement or disagreement.

Interpretation. Unexpected results should trigger checks for leakage, class imbalance, preprocessing mismatch, random-seed effects and hyperparameter choices. They should not be hidden or adjusted merely to match theory. Scientific reporting gains credibility when limitations and contradictory observations are discussed openly.

Theory	Measurement	Interpretation
PCA reduces dimensions	Macro-F1 and size	Trade-off
Scaling aids conditioning	Loss/time curve	Convergence
High capacity raises variance	Train/val gap	Generalization

```
# Compare theory with measurements, not
assumptions print('PCA variance:',
pca.explained_variance_ratio_.sum())
print('Macro-F1:', measured_macro_f1)
print('Latency (ms):', measured_latency_ms)
```



TASK 11 — Is deep learning justified?

CNNs can learn local edges, textures and hierarchical visual features that flattened pixels and classical baselines may not represent efficiently. k-NN is hindered by high-dimensional distance; linear models may miss nonlinear patterns; tree ensembles can model interactions but may require many splits. Deep learning is justified only if a carefully tuned CNN produces a meaningful validated improvement in macro-F1, minority-class recall or robustness, and its deployment cost is acceptable. Transfer learning can reduce labelled-data requirements, but domain mismatch must be checked. Saliency maps can help identify whether predictions rely on lesions rather than backgrounds. Compare quality with training cost, inference latency, memory, energy and interpretability. If a classical model meets the target constraints, its simplicity may be preferable. The example chart is explicitly illustrative and must not be described as measured evidence. The final decision should follow the actual baseline table and deployment measurements.

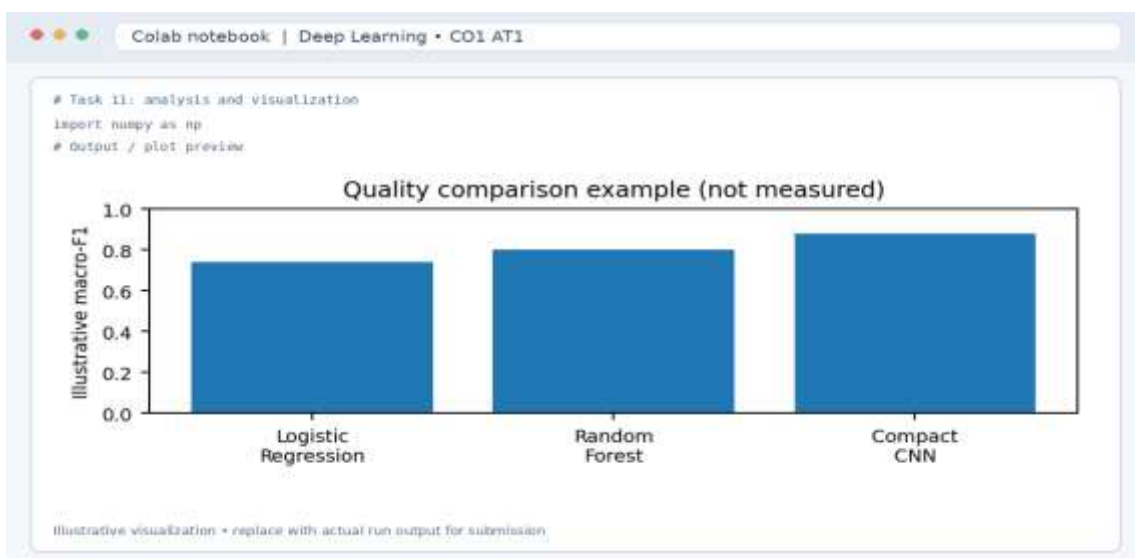
Decision evidence

Method and analysis. Deep learning should be compared against tuned classical baselines rather than assumed to be superior. A compact CNN can learn spatial filters that use neighbouring pixels, while a classical model on engineered or compressed features may be cheaper and easier to explain.

Interpretation. The decision depends on validated quality and operating constraints. In addition to macro-F1, evaluate minority-class recall, robustness to lighting and backgrounds, end-to-end latency, model size, memory and interpretability. If a simpler model meets the requirements, it may be the more appropriate engineering choice.

Criterion	Measure	Decision value
Quality	Macro-F1 / recall	Class performance
Efficiency	Latency / size	Deployment fit
Trust	Error analysis	Reliability

```
quality_ok = cnn_macro_f1 >
best_baseline_macro_f1
latency_ok =
latency_ms <= 50
size_ok = model_size_mb <= 5
print(quality_ok, latency_ok,
size_ok)
```



TASK 12 — Changed requirements and design challenge

The revised system must handle 38 classes, approximately 20% noisy labels, latency below 50 ms and a model size below 5 MB. With 38 equally likely classes, maximum entropy is $\log_2(38) \approx 5.248$ bits; additional classes create more confusion opportunities. Noisy labels can teach incorrect targets, raise variance and distort evaluation. The original design should therefore be modified unless measurements show that it already satisfies the new limits. Measure end-to-end latency on the target phone and distinguish serialized model size from runtime memory. Possible adaptations include reducing PCA dimension, stronger regularization, label smoothing or noise-robust loss, quantization, pruning and a compact CNN. Lower PCA dimension reduces cost but may remove disease cues; stronger smoothing can help noisy labels but may weaken confidence; quantization reduces size but can alter predictions. Compare all options using macro-F1, per-class recall, latency and size. The graph below uses synthetic values solely to illustrate the decision process.

Constraint comparison — illustrative only

Method and analysis. The revised constraints must be treated as hard acceptance criteria. A model with strong classification results is not deployable if it exceeds the latency or storage limit. Measure the complete pipeline on the target phone, including resizing, normalization and model invocation.

Interpretation. Each adaptation has trade-offs. PCA can reduce input size but discard subtle cues; quantization reduces weight precision and file size but can change predictions; label smoothing may reduce overconfidence but can also soften useful class distinctions. Evaluate every option on the same held-out data and document the quality-cost trade-off.

Candidate	Latency	Size	Status
Baseline	Measure	Measure	Check limits
PCA / regularization	Measure	Measure	Trade-off
Quantized compact CNN	Measure	Measure	Trade-off

```
latency_ok = latency_ms
<= 50    size_ok    =
model_size_mb <= 5
    print('Latency meets limit:',
latency_ok) print('Model size
meets limit:', size_ok)
```

